AarogyaNet

Rural Healthcare Intelligence Platform

Complete Technical Report

Document Information

Date:	26 May 2026
Commit:	b05dea6d65d8851d10e9e6e6f7790d99c2c1a08d
Stack:	Flask 3.x · PostgreSQL · XGBoost · SHAP · Blockchain (SHA-256/ECDSA/PoW)
Status:	Active development -- 23 features implemented

13	40+	8	1	23
Blueprints	API Endpoints	DB Tables	Trained ML Model	Features Built

AarogyaNet is a rural healthcare intelligence platform designed for the Indian public health system. It digitises and augments the clinical workflows of ASHA workers, doctors, and patients managing hypertension, integrating a blockchain-secured visit ledger, a guided clinical protocol engine (GCPE), longitudinal patient memory, disease cluster detection, and an explainable XGBoost diabetes risk model -- all in a single offline-first Flask application.

TABLE OF CONTENTS

1. Project Overview

AarogyaNet is a rural healthcare intelligence platform designed for the Indian public health system. It digitises and augments the clinical workflows of three primary user roles across hypertension management:

* ASHA Workers (Accredited Social Health Activists) -- conduct home visits, record vitals, complete guided clinical assessments via a step-by-step protocol wizard, and raise SOS emergencies.
* Doctors -- review a risk-ranked patient queue, analyse longitudinal trends, issue prescriptions, and acknowledge SOS emergencies with triage context.
* Patients -- view their own health records and prescription token redemption history.

Core problems solved:

* Paper-based ASHA visit records are unstructured and non-actionable.
* Doctors in rural CHCs/PHCs lack prioritised, longitudinal intelligence across large patient populations.
* Medication adherence has no verifiable audit trail.
* Outbreak signals are invisible until clusters become crises.
* No ML-based early disease risk stratification in the primary care workflow.

The application is an offline-first, blockchain-secured, AI-augmented clinical decision support system. All AI outputs are explicitly framed as augmentation signals -- the deterministic rule engine always takes precedence for safety-critical decisions. The platform is built on Flask 3.x with a PostgreSQL primary database and SQLite offline queue, deployed on a single Python process.

2. Tech Stack

Backend Framework

Component	Library / Package	Version
>= 3.0.0		
>= 4.0.0		
>= 4.6.0		
>= 21.2.0		
>= 1.0.0		

Database

Component	Technology	Version
>= 2.9.9		
Python stdlib		

AI / Machine Learning

Component	Library	Version
3.2.0		
0.51.0		
>= 1.4.0		
>= 2.2.0		
>= 1.26.0		
bundled		

Blockchain / Cryptography

Component	Implementation	
SHA-256 (stdlib hashlib)		
ECDSA via cryptography >= 42.0.0		

Proof-of-Work

Custom Python mining loop (difficulty=4, 4 leading zeros)

Key Storage

ECDSA keypair: keys/ecdsa_private.pem + keys/ecdsa_public.pem

Frontend

Component	Technology
Jinja2 (Flask-native server-side rendering)	
Bootstrap 5 (CDN)	
Font Awesome 6 (CDN)	
Leaflet.js (CDN) + OpenStreetMap tiles	
Chart.js (CDN)	
Vanilla ES6+ (no build step, no bundler)	

3. Project Structure

Root Level

File / Directory	Description
Flask app factory; registers 13 blueprints + 11 page routes	
Config class: DB URLs, JWT secret, PoW difficulty, port	
Python package dependencies	
SQLite file for offline emergency queue	
ECDSA private + public PEM keypair for blockchain signing	

ai_engine/ -- AI & ML Infrastructure

File	Description
PROTECTED -- visit feature extraction for the deterministic risk engine	
PROTECTED -- 3-layer clinical risk scoring (vitals + GCPE + guards)	
Raw CSV datasets: Pima Indians (768r), Heart Disease UCI, NFHS-5	
DATASET_REGISTRY -- schema definitions for all 5 datasets	
DatasetProfiler -- statistical profiling with JSON/text report output	
AarogyaModelBase -- abstract base: save/load/predict contract	
ModelRegistry singleton -- register, load, cache, introspect models	
Saved joblib models + diabetes_eval_report.json	
DiabetesRiskModel, predict_diabetes_risk(), get_model_status()	
PimaPreprocessor -- zeros->NaN, median impute, StandardScaler	
DiabetesTrainer -- full train->evaluate->SHAP->save pipeline	
DiabetesExplainer -- SHAP TreeExplainer (tree_path_dependent)	
FeaturePipeline -- extract 35 features from visit data	
FeatureStore -- JSON-backed feature vector cache	
evaluate_classifier() -- AUC, F1, sensitivity, Brier, ECE	

Other Top-level Directories

Directory	Contents
blockchain.py, block.py, hashing.py, mining.py, verification.py	
postgres.py (init_db, get_db_connection), sqlite_sync.py	
engine.py + protocols/hypertension_protocol.json (15-step branching protocol)	
13 Flask blueprints -- one file per domain (auth, patients, visits, risk, ...)	
8 business logic modules -- longitudinal, emergency, cluster, absence, tokens, ...	
auth.py (hash/check password), helpers.py (hash, serialise), validators.py	
css/style.css (Bootstrap overrides), js/main.js (JWT + API singleton)	
10 Jinja2 templates: base, login, index, asha, doctor, patient, blockchain, ...	

4. All Features Implemented

#	Feature	Description	Status
Complete		in; dual storage (header + cookie); 24h expiry; role claims	
Complete		., doctor, patient, admin; enforced on every endpoint	
Complete		ers patients with demographics, GPS, phone, medical history	
Complete		mp, symptoms, GPS + timestamp; SHA-256 tamper-hash	
Complete		ching hypertension assessment wizard; pathway-aware	
Complete		· GCPE pathway boosts + clinical contradiction guards	
Complete		+ ECDSA + PoW (difficulty=4) for every ıcy/adherence	
Complete		ed by risk score descending with latest vitals	
Complete		te prescriptions; JanAushadi token per medication	
Complete		A redeems token; method mined to blockchain	
Complete		SOS; triage instructions; doctor notified; blockchain proof	
Complete		s SOS when offline; /offline/sync replays to PostgreSQL	
Complete		sed visits; continuity score; LAPSED/AT_RISK alerts	
Complete		‹ trajectory, adherence %, first-deviations, narrative	
Complete		ıatiotemporal; 6 cluster types; CRITICAL/HIGH/MEDIUM	
Complete		oured GPS markers; village stats overlay	
Complete		ofiling of 5 public health datasets with reports	
Complete		ry, AarogyaModelBase, 35-feature pipeline, feature store	
Complete		JC=0.85) on Pima Indians; SHAP explainability; 4 risk bands	
Complete		gated panel: risk band, SHAP factors, completeness %,	
Complete		w, block lookup, integrity verification, stats	
Complete		er view, type summary, village breakdown	
Complete		visit history, prescriptions, token adherence timeline	

5. Key Functions & Methods

ai_engine/risk_engine.py

Function	Signature	What It Does
3-layer pipeline: vitals rules -> GCPE pathway boosts -> clinical contradiction guards		
Structured clinical summary: escalation reasons, adherence concerns, guard notes, referral urgency		
Normalises GCPE fields; backward-compatible for pre-GCPE visits		
LOW (<21) / MEDIUM (21-50) / HIGH (>=51) classification		
Normal / Stage1 / Stage2 / Crisis classification string		

ai_engine/models/diabetes/model.py

Function	Signature	What It Does
Load model -> preprocess -> XGBoost predict -> SHAP explain -> format 12-field output		
Artifact existence, version, AUC-ROC, sensitivity, model path		
Raw model inference with risk band + 5 contributing SHAP factors		
joblib serialisation to .joblib artifact		
joblib deserialisation with preprocessor loading		

services/longitudinal_engine.py

Function	Signature	What It Does
Full PCM: BP trend + risk trajectory + adherence + deviations + pathways + alerts + narrative		
Lightweight: trend directions + top 3 alerts only (fast for list views)		
IMPROVING/WORSENING/VOLATILE/STABLE/SINGLE with baseline, recent, delta, std		
ESCALATING/IMPROVING/STABLE with baseline/recent/peak risk scores		
Adherence %, trend, consecutive misses, top missed reason		
7 clinically significant first-occurrence events with severity and description		
Visits where risk score jumped >=20 points from previous visit		
Prioritised human-readable alert strings for the doctor dashboard		

services/cluster_engine.py

Function	Signature	What It Does
Runs all 6 cluster type detectors; returns severity-sorted cluster list		

get_surveillance_summary(lookback_days)

(int) -> dict

District-level summary: total clusters, type breakdown, village breakdown

_group_into_clusters(...)

(...) -> list

Greedy Haversine spatiotemporal grouping (radius_m + window_days constraints)

_haversine_m(lat1, lon1, lat2, lon2)

(float×4) -> float

Distance in metres between two GPS coordinates using Haversine formula

_classify_severity(ctype, count)

(str, int) -> str

CRITICAL/HIGH/MEDIUM based on cluster type and case count

_cluster_id(ctype, lat, lng, dt)

(...) -> str

Deterministic SHA-256 cluster fingerprint (stable across re-runs)

blockchain/blockchain.py

Function	Signature	What It Does
Hash payload -> mine PoW -> ECDSA sign -> persist to blockchain table		
Same pipeline for emergency SOS events		
Same pipeline for token redemption adherence events		
Rehash + re-verify ECDSA signatures across entire chain; returns integrity report		
All blocks from DB, newest first		

gcpe/engine.py

Function	Signature	What It Does
Load JSON protocol definition by ID from gcpe/protocols/		
Evaluate answer conditions -> return next step object or None if complete		
Compile completed GCPE session into structured clinical summary		
Type-check answer against step type (boolean/choice/text/number)		

6. Database / Data Models

All tables are created in PostgreSQL by database/postgres.py on first startup. SQLite mirrors the emergency_events schema for offline queue storage.

users

Column	Type	Description
Auto-increment primary key		
Login username		
Email address		
SHA-256 + salt hashed password		
asha | doctor | patient | admin		
Display name		
Contact number		
Assigned village		
Account active flag		
Account creation time		

patients

Column	Type	Description
Auto-increment primary key		
AAR-XXXXXXXX format UID		
Patient full name		
Age in years		
Gender		
Home village		
Contact number		
Assigned ASHA worker		
Assigned doctor		
Free-text medical background		
Home GPS coordinates		
Updated after each visit		

asha_visits

Column	Type	Description
Auto-increment primary key		
Patient reference		
Recording ASHA worker		
Visit timestamp		
Unix epoch for replay protection in hash		
Blood pressure readings mmHg		
Heart rate bpm		
Body temperature °C		
Symptom flags		
Free-text notes		
0-100 computed risk score		
LOW | MEDIUM | HIGH		
Full GCPE session: pathways, answers, alerts, risk_flags		
SHA-256 tamper-evidence hash		
Linked blockchain block index		
Blockchain mining flag		
Visit GPS coordinates		

blockchain

Column	Type	Description
Sequential block index		
Unix epoch of block creation		
PHI-stripped block payload		
Hash of preceding block		
SHA-256 hash of this block		
Proof-of-work nonce		
ECDSA base64 signature		
Verification flag		
visit | emergency | adherence		

emergency_events

Column	Type	Description
UUID hex event identifier		
Patient in distress		
ASHA who raised the SOS		
hypertensive_crisis | chest_pain | unconscious | ...		
active | acknowledged | resolved | escalated		
CRITICAL | HIGH		
Emergency location GPS		
Most recent BP from visit history		
Doctor, PHC contact, helpline, supervisor		
Step-by-step first-response instructions		
SHA-256 of event_id + type + timestamp		
Linked blockchain block		
Acknowledging doctor ID		
Lifecycle timestamps		

prescriptions & prescription_tokens

Column	Type	Description
Prescription ID		
List of medicine name strings		
Dosage free text		
Scheduled follow-up		
Hex redemption token		
pending | redeemed | expired		
patient_pickup | asha_pickup | home_delivery		
SHA-256 of redemption event		
Linked blockchain block		

absence_events

Column	Type	Description
Auto-increment primary key		
Patient reference		
ASHA/doctor who recorded absence		
missed_visit | patient_unavailable | refused_visit | ...		
When the visit was expected		
Reason/context		
Record timestamp		

7. API Endpoints

Authentication -- /api/auth

Method	Path	Roles	Description
	Authenticate; returns JWT + sets cookie		
	Register new user		
	Current user profile		
	List all users		

Patients -- /api/patients

Method	Path	Roles	Description
	List patients (role-scoped)		
	Register new patient		
	Patient + visits + prescriptions		
	Own patient record		
	Full PCM summary		
	Lightweight trend snapshot		

Visits -- /api/visits

Method	Path	Roles	Description
	Create visit -> risk score -> mine block		
	List visits (role-scoped, up to 100)		
	Single visit + risk summary + longitudinal snapshot		
	All visits for one patient		

Risk -- /api/risk

Method	Path	Roles	Description
	Preview risk score (no DB write)		
	Risk-ranked patient queue		
	Queue with longitudinal trend data		
	GPS points for Leaflet map		
	Aggregate dashboard stats		

Blockchain -- /api/blockchain

Method	Path	Roles	Description
	Full chain, newest first		
	Single block by index		
	Full integrity report		
	Most recent block		
	Chain summary statistics		

GCPE -- /api/gcpe

Method	Path	Roles	Description
	Full protocol JSON definition		

POST

/api/gcpe/next

JWT any

Advance one step; returns next step + risk flags	

POST

/api/gcpe/summary

JWT any

Build summary from completed session

Prescriptions -- /api/prescriptions

Method	Path	Roles	Description
	Create prescription + generate tokens		
	List prescriptions (role-scoped)		
	Single prescription detail		
	Patient prescriptions		

Tokens -- /api/tokens

Method	Path	Roles	Description
	Redeem token; mine adherence block		
	Token status and redemption info		
	Tokens for a prescription		
	Tokens for a patient		
	Role-aware token listing		

Emergency -- /api/emergency

Method	Path	Roles	Description
	Available emergency type enum (offline-safe)		
	Create SOS + triage + blockchain proof		
	List emergencies (role-scoped)		
	Single event detail with patient info		
	Doctor acknowledges SOS edge		
	Mark SOS resolved		
	All events for a patient		
	SQLite offline queue count		
	Replay SQLite queue to PostgreSQL		

Absence -- /api/absence

Method	Path	Roles	Description
	Absence type enum		
	Record missed visit event		
	Absence history + continuity analysis		
	All patients with active absence alerts		
	Continuity overview table		

Clusters -- /api/clusters

Method	Path	Roles	Description
	Active clusters (?lookback_days, ?type, ?severity)		
	District surveillance summary		
	Cluster overlay for Leaflet.js		

POST

/api/clusters/recalculate

JWT asha/doc/admin

Force re-detection after offline sync

AI Inference -- /api/ai

Method	Path	Roles	Description
	XGBoost inference; all 8 fields optional (imputed if missing)		
	Model registry + artifact status + phase		
	Global SHAP feature importance + eval metrics		

8. UI Components

base.html *Shared layout*

Navbar with role-aware links, Bootstrap 5 + Font Awesome 6 CDN, main.js + style.css includes, {% block content %} and {% block scripts %} extension points.

login.html *Login page*

Username/password form. JWT stored to localStorage on success. Redirects to role-appropriate dashboard (/asha, /doctor, /patient).

asha_dashboard.html *ASHA Dashboard*

Patient list with search/filter, multi-step visit wizard (vitals -> GCPE wizard -> risk preview -> submit), SOS panel with emergency type selector + GPS capture, offline queue display.

doctor_dashboard.html *Doctor Dashboard*

Risk queue with longitudinal trend badges (ESCALATING), patient detail panel, AI Diabetes Signal panel (confidence-gated >=0.70), PCM longitudinal view (Chart.js BP trend), prescription form, token/adherence history, visit history (last 5).

patient_dashboard.html *Patient Dashboard*

Own health record, latest vitals, risk level, prescription list, token redemption status with blockchain hash display.

blockchain.html *Blockchain Explorer*

Full chain table (block index, type, hash, timestamp, validity badge), block lookup form, integrity verification button, chain stats card.

heatmap.html *Risk Heatmap*

Leaflet.js map centred on Rajasthan, coloured risk markers (red=HIGH/amber=MEDIUM/green=LOW), cluster overlay from /api/clusters/heatmap, village stats sidebar.

longitudinal.html *Longitudinal PCM View*

BP trend chart (Chart.js line), risk trajectory chart, first-deviations timeline, GCPE pathway history table.

surveillance.html *Surveillance Dashboard*

District cluster summary cards (CRITICAL/HIGH/MEDIUM counts), active clusters table (type, severity, village, case count), cluster type breakdown, village breakdown table.

9. State Management

State	Storage Location	How It Flows
Sent as Authorization: Bearer <token> header on every API call		
Extracted server-side via get_jwt() and get_jwt_identity()		
Set on login; used only for Jinja2 template current_user injection		
Accumulated answers dict passed in each /api/gcpe/next request body		
Accumulated across multi-step wizard; submitted atomically on completion		
Written when PostgreSQL unreachable; synced via /api/emergency/offline/sync		
Lazy-loaded on first inference call; cached for entire process lifetime		

10. Integrations

Integration	Status	Notes
Via DATABASE_URL env var; Replit-managed; all clinical data		
Local aarogyanet_offline.db; emergency offline queue		

ECDSA Keys

Active

Pre-generated PEM keypair in keys/; blockchain block signing

Leaflet.js / OpenStreetMap

Active

CDN; map tiles for heatmap + cluster overlay pages

Bootstrap 5 + Font Awesome 6

Active

CDN; all UI styling and icons

JanAushadi Pharmacy

Mock

Prescription tokens named after JanAushadi; no real API

National Emergency Helpline (112)

Mock

Hardcoded string in EMERGENCY_HELPLINE constant

PHC Contact Directory

Mock

3 PHCs hardcoded in MOCK_PHC_CONTACTS dict

SMS / Push Notifications

Mock

notification_service.py prints to console only

ICMR / NFHS-5 / HMIS Datasets

Profiled

Loaded and profiled; not yet used for model training

11. Authentication & Security

Authentication Architecture

* JWT via flask-jwt-extended; 24-hour expiry; stored in localStorage and HTTP-only cookie aarogyanet_token
* Token accepted from both Authorization: Bearer header and cookie -- dual-mode for browser + API client support
* Password hashing via utils/auth.py using SHA-256 + salt (note: not bcrypt -- weaker than industry standard)
* JWT_COOKIE_CSRF_PROTECT = False (dev mode)

Role-Based Access Control

Every API endpoint enforces role checks via get_jwt()['role']. The four roles and their permissions:

Role	Permissions
	Own patients only; create visits; raise SOS; record absences
	All patients; risk queue; prescriptions; acknowledge/resolve emergencies
	Own record only; view prescriptions and tokens
	Full access across all endpoints

Clinical Data Integrity

* Every visit payload is SHA-256 hashed (visit_hash) before DB insert -- tampering breaks the hash
* Visit hash includes GPS coordinates + Unix timestamp -> replay-attack protection
* Every visit, emergency, and adherence event is mined into blockchain (SHA-256 + ECDSA + PoW difficulty=4)
* Chain can be fully verified at any time via GET /api/blockchain/verify

AI Safety Properties

* deterministic_override field present in every AI API output -- states that the clinical rules engine takes precedence
* No diagnosis language used; only "clinical evaluation advised" phrasing
* Doctor dashboard AI signal hidden when model confidence < 70% (confidence_above_threshold: false)
* All 8 Pima feature names verified PHI-free (no names, phone, Aadhaar, address, GPS, email)

Known Security Gaps (Development Mode)

* CSRF protection disabled (JWT_COOKIE_CSRF_PROTECT = False)
* CORS allows all origins (CORS_ORIGINS = ['*'])
* Registration endpoint open -- no admin-only gate in current deployment
* JWT secret falls back to hardcoded string if SESSION_SECRET env var not set

12. What Is Not Yet Built

Planned ML Models (status = "planned" in registry)

Model Key	Algorithm	Dataset	Target
Predict future BP systolic from longitudinal visit history			
Village-level outbreak probability from HMIS patterns			
Likelihood of next-visit medication miss			

Feature Gaps Identified from Code Analysis

* Real notifications -- notification_service.py only prints to console; no SMS, push, or WhatsApp delivery
* Real PHC directory -- 3 hardcoded entries; no integration with government health facility registry
* ASHA supervisor role -- referenced in escalation targets but no supervisor user type or dashboard exists
* Patient-level diabetes risk history -- AI predictions are stateless (not stored in DB against patient record)
* Doctor-to-patient direct messaging -- no messaging system despite notification infrastructure hooks

* Patient appointment scheduling -- followup_date field exists in prescriptions but no scheduling feature
* GPS auto-capture in visit wizard -- browser Geolocation API referenced but not fully wired
* Heart disease risk model -- UCI dataset profiled and present in raw/ but no model built
* NFHS-5 / HMIS dataset models -- datasets profiled; schemas defined; no training pipelines yet
* Admin dashboard -- admin role has full access but no dedicated admin UI
* Offline visit recording -- SQLite handles offline emergencies but not offline visits
* Multi-language support -- all UI in English; platform targets Hindi-speaking ASHA workers
* Report / export feature -- no PDF generation for prescriptions or patient summaries
* Additional GCPE protocols -- only hypertension_protocol.json exists; diabetes, COPD absent

13. Known Issues & Incomplete Areas

Browser Console Errors (Pre-existing)

```
allQueue.filter is not a function        (doctor_dashboard.html:430)
(alerts || []).filter is not a function  (doctor_dashboard.html:854)
Dashboard load error: {}
```

These fire when the doctor dashboard loads before authentication resolves. The API returns a non-array response and .filter() is called without defensive type-checking.

Code-Level Issues

File	Issue
flask listed three times (lines 1, 11, 12) -- redundant duplicates	
SECRET_KEY and JWT_SECRET_KEY both fall back to the same hardcoded string if env var unset	
Uses SHA-256 + salt, not bcrypt -- passwords are less secure than industry standard	
All notification dispatch is print() calls only -- no real delivery mechanism	
PHC contacts hardcoded for 3 villages; all others get a single default entry	
GET /api/tokens/my queries patients WHERE doctor_id=%s but patients table has no doctor_id column	
loadDiabetesSignal() submits mostly zero-valued features (glucose=0, BMI=0) -- confidence threshold rarely met	
No connection pooling -- each request opens and closes a new psycopg2 connection	
PoW is synchronous -- high-BP visits under load could increase response latency	
Root models/ package is empty -- leftover SQLAlchemy scaffolding, not used	

Hardcoded Demo Data

* MOCK_PHC_CONTACTS: 3 hardcoded PHC names and phone numbers (Rampur, Sundarpur, Bhatpur)
* EMERGENCY_HELPLINE: hardcoded "National Emergency -- 112"
* ASHA supervisor contact: hardcoded 9800000000 in all escalation targets
* Demo credentials: admin/admin123, dr_rajan/doc123, asha_priya/asha123, patient_ram/pat123

Appendix: Diabetes Risk Model Performance (v1.0.0)

Trained on Pima Indians Diabetes Database (768 rows, 8 features, 70/15/15 split):

0.8503	82.5%	77.6%	0.7333	0.154
AUC-ROC	Sensitivity	Specificity	F1 Score	Brier Score

SHAP Global Feature Importance